Solution Tutorial 1+2

1. **Numerical errors**

Let's look at the (unnormalised) sinc function $\mathrm{sinc}(x)$, defined as

$$\mathrm{sinc}(x) = \frac{\sin(x)}{x}. \tag{1}$$

Its power series expansion around zero is given by

$$\mathrm{sinc}(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n+1)!} = 1 - \frac{x^2}{3!} + \frac{x^4}{5!} - \frac{x^6}{7!} + \cdots, \tag{2}$$

where '!' denotes the factorial function.

(a) Write a piece of code that implements the $\mathrm{sinc}(x)$ function in two ways: one using the power series expansion, and one using a library function, e.g. np.sin(x). Clearly, the accuracy of the answer given by the power series expansion depends on the number of terms included. Which kind of error are we dealing with in such an example?

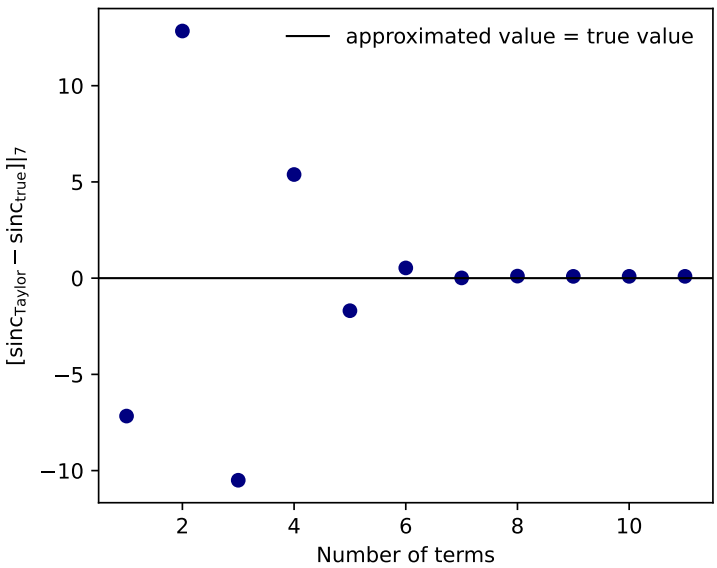
(b) Compare the two implementations for a different number of terms. Let's look at the value of sinc(7). How does the error change when more terms are added? Does the error accumulate preferentially in one direction, or does it oscillate?

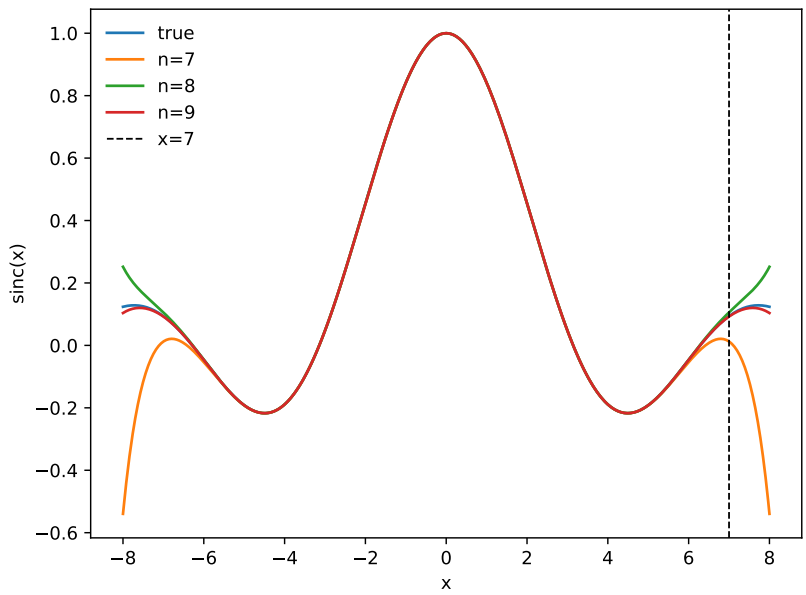
Solution:

For this exercise, note that `numpy.sinc(x)` returns the *normalised* sinc function, and we will work with the unnormalised one, as defined by equation 2.

(a) The accuracy of the answer given by the Taylor expanded sinc function is only an approximation of the function around zero, and depends on the number of terms - the less terms we include, the less accurate the answer would be, having a larger *truncation* error.

(b) The plot below shows the truncation error for an increasing number of terms. We can see that including less than 6 terms results in a very wrong estimate of sinc(7), but for $n > 6$ the error gets smaller. In absolute terms, the error first increases, and then decreases with larger n, and its sign oscillates, which makes sense given the alternating plus and minus signs in the Taylor expansion (eq. 2). Below you will also find a plot of $\mathrm{sinc}(x)$ for different n over the domain $x \in [-8, 8]$.





For $n > 6$, the factorial overflows in single precision. Therefore, $2n + 1 = 13$ overflows in double precision. The maximal accuracy we can reach in single precision is about $\approx 10^{-7}$. When computing $\mathrm{sinc}(x)$ in both single and double precision, a slight discrepancy appears around $n = 6$ due to the roundoff error. Recalling the previous part, note that this means in single precision, the value of $\mathrm{sinc}(x)$ for $x > 7$ in such an unoptimised single precision setting will give a wrong answer.

2. **A short timing test**

Suppose that you need to calculate the Schwarzschild radius R_S of a distribution of black holes of mass $M_\bullet$. R_S is given by

$$R_S = \frac{2GM_\bullet}{c^2}, \tag{3}$$

where $G = 6.67 \cdot 10^{-11}\ \frac{\mathrm{m}^3}{\mathrm{kg}\cdot\mathrm{s}^2}$ is Newton's gravitational constant and $c = 3 \cdot 10^8\ \frac{\mathrm{m}}{\mathrm{s}}$ is the speed of light.

(a) Generate the masses of 10000 supermassive black holes by drawing from a Gaussian distribution with mean $10^6\ \mathrm{M}_\odot$ and standard deviation $10^5\ \mathrm{M}_\odot$ (you can use e.g. np.random.normal).

(b) Using the `timeit` module, calculate the Schwarzchild radius of the black holes directly as given by equation 3.

(c) Calculate R_S and use `timeit` again, but this time instead of dividing by c^2, multiply by a predefined variable `c_inv2 = c_inv * c_inv`, where `c_inv = 1/c`. Compare the results.

Solution:

You may safely ignore units in this exercise. Here you are told that you can use `np.random.normal` with parameters (`loc=1e6, scale = 1e5, size=10000`), but note that in future lectures you will learn how to sample different distributions, and may be expected to write the code for this yourself.

The time it takes for 1 000 000 iterations (the default `timeit` setup) of the calculation for both setups will vary depending on the device that you use, but overall there is about 40% reduction in time when you multiply by $(1/c) \times (1/c)$ instead of straight dividing by c^2. In fact, when

writing the subgrid models for simulations on galaxy formation, taking care of these numerical subtleties can make a big difference in the execution time.

3. **Image interpolation**
An application of interpolation is image resizing (*rebinning*). When expanding the size of an image, the values for new pixels can be obtained by interpolating the values of the original image in 2-steps (such as bilinear or bicubic interpolation).

For this exercise, take `https://home.strw.leidenuniv.nl/~daalen/NURfiles/M42_128.jpg` or any black and white image of your choice. Read the image using
`from matplotlib.image import imread`, followed by `image=imread(''M42_128.jpg'')`, which will load in the image as a numpy array with pixel values.

(a) Write a linear interpolator and apply it to the first row of the image for 201 equally spaced points. Implement bisection to find the enclosing two grid points for each x you want to interpolate at. Plot both the measurements from the image and the interpolated results.

(b) Now write a polynomial interpolator (Neville's algorithm) and overplot these results as well.

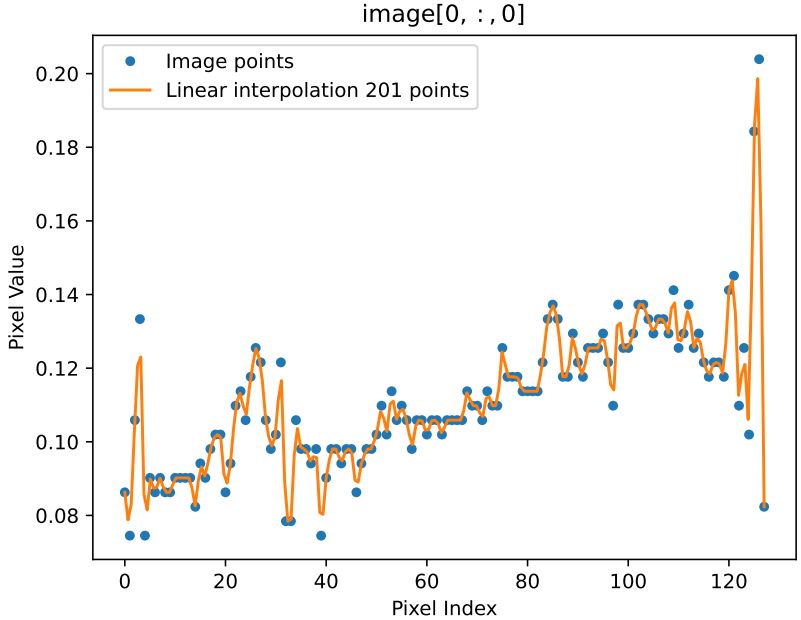
(c) Expand each of the dimensions of the image by a factor 2 using any two of the interpolation methods you have seen in the lecture as your base for 2-step interpolation – or, use true bilinear interpolation as shown in the lecture. Compare the times the different methods took (you may have to run them many times in a row to get a meaningful run time). Which one is better and why? Can you suggest any other method to enlarge an image? Is the total flux density conserved? How can you ensure flux density conservation?

Solution:

(a)
1. Read in the image and convert it into numpy array.
2. Get the dimensions of image.
3. Implement bisection. Be aware of boundaries. You can use `for` or `while` loops.
4. Implement linear interpolation as:

$$y = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i) + y_i,$$

where (x_i, x_{i+1}) are the left and right neighbours of x found using bisection.



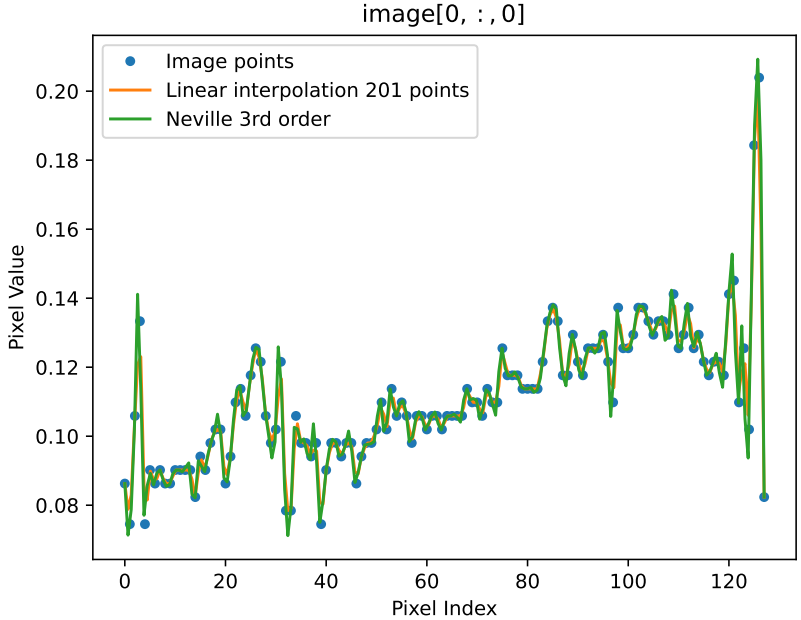
(b) Use slide 9 of Lecture 2 as a reference. Before starting the interpolation, check that the index of the left neighbour of the interpolation point is non-negative and does not exceed $N - M$, where N is the total number of points and $M - 1$ is the interpolation order. This guarantees that there are M data points for the interpolation.

This array stores the intermediate results of the Neville scheme and is updated in place. For memory efficiency, use the one-dimensional array of length M filled with the values of the selected M data points used for interpolation. The algorithm applies two nested `for` loops. At each step, the array entries are updated as

$$H_i(x) = \frac{(x_{i+k} - x)F_i(x) + (x - x_i)F_{i+1}}{x_{i+k} - x_i},$$

where $F_i(x)$ are the values of the $H_i(x)$ from the previous iteration.

You can notice that Neville's interpolated curve lies closer to the original data points.



(c)

1. Create a new 2-D array, with each dimension equal to 2× the original dimension and store it into a new variable (e.g., `img2`).
2. Copy the original image into the new array at every other location, i.e., element (0,0) will go to (0,0); (0,1)->(0,2); (1,0)->(2,0), etc. For this step, you can use two nested `for` loops, one per dimension. This is quite inefficient, though, especially when matrices grow in size. Alternatively, you can create two new matrices (one for the x axis, and another one for the y axis) that will contain the indexes of img2 where the original image data will be copied to.
3. Now with a for loop you can apply the linear interpolation on rows to find the values at those intermediate locations which are bracketed by values from img; repeat the procedure looping over columns to fill in the remaining values. Values corresponding to the last row and last column can not be obtained via interpolation, but linear extrapolation is required.

If instead you want to apply bilinear interpolation, you could proceed as follows:

1. Identify the 4 neighboring pixels whose values constitute the reference for computing the interpolation. Given we are dealing with an image, their coordinates will be integer, so one can use floor(new_x) and floor(new_y) for the *leftside/lower* coordinate, and adding +1 for the *rightside/upper* coordinates.
2. Next, extract from the original image the values at the 4 corresponding neighboring pixels, storing them into 4 distinct variables. Beware that matrices work as `A[row,column]` so you need to switch indexes to get the values at the specified x and y, i.e., you will need `image at x,y=A[y,x]`.
3. Compute the four slopes.
4. Finally linear-combine the values of the neighboring pixels and corresponding slopes. This will give you the values at the interpolating locations, e.g.

$$u = (x - x_i)/(x_{i+1} - x_i)$$
$$v = (y - y_i)/(y_{i+1} - y_i)$$
$$A(x, y) = A_{i,i}(1-u)(1-v) + A_{i+1,i}u(1-v) + A_{i,i+1}(1-u)v + A_{i+1,i+1}uv.$$

Beware of $x_i = x_{i+1}$ or $y_i = y_{i+1}$, where you need to modify the expression above.

5. You can call the above procedure from a double for-loop, looping over individual x and y values. However, note that if inside the procedure you store the variables as suggested, the above procedure can be immediately applied to all interpolating points in a specific row. It would also be possible to run the procedure only twice. The first time, giving in input the locations at even rows, the second time the odd rows (which by construction have a larger number of unknown values than the even rows). For this to work, you first need to define the full set of interpolating locations for each one of the two sets of locations.
6. Finally, substitute the interpolated values into the matrix.

The bilinear interpolation will give the same results as the double application of the linear interpolation, but bilinear is faster as it computes the result in one step only. Because we are adding data points to the original image, the resampled image will not preserve the flux. This can be verified by computing the total flux density of the input image and comparing it to that of the resized image. The total flux density in the enlarged image will be larger by a factor `(new_x/old_x)`$\times$`(new_y/old_y)`, where `new_x,y` are the x and y size of the enlarged image, while `old_x,y` are the x and y size of the original image. One way to ensure flux density conservation is to renormalize the image entirely. Another option is to conserve flux on a per-pixel basis, renormalizing every square of four pixels such that their total flux adds up to the flux of the original pixel before it was subdivided.

4. **Compute the area of a polygon**
Suppose we have some polygon with vertices specified by the coordinates (x_0, y_0), (x_1, y_1),...,(x_{n-1}, y_{n-1}), numbered either in a clockwise or counter clockwise fashion around the polygon. If the polygon is non-self-intersecting (i.e. it is simple, does not intersect itself, it has no holes, etc), the area of the polygon can be computed by just knowing the boundary coordinates:

$$A = \frac{1}{2}\left|\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)\right| \tag{4}$$

(a) Write a function `polygon_area(x, y)` that takes two coordinate lists with the vertices as arguments and returns the area. For this function, use for loops to operate over each entry in the array every time. Test the function on a triangle, a quadrilateral, and a pentagon where you can calculate the area by alternative methods for comparison. Convinced yourself that your function returns the correct area of some regular pentagons.

(b) Vectorize equation (4) such that there are no Python loops in the implementation. Make a test function that compares the scalar implementation with the new vectorized implementation for some chosen polygons. *Hint: `numpy.dot` could be useful to vectorize the code.*

(c) Use `timeit` to compute how much time your code takes to compute all the areas using the vectorized implementation versus the for-loop implementation. Create 10,000 polygons with the number of vertices N, using `numpy.random` to generate x and y arrays. Run your code using the scalar for-loop and vectorized implementations and compare the time that it takes to finish. Vary N from 3 to 20, repeating the procedure. When is the vectorised implementation faster? Is it always faster?

Solution:

There are several ways to vectorize this code and make it efficient. One example is by realizing that the term $\sum_{i=0}^{n-1} x_i y_{i+1}$ is the dot product of two vectors, x[:-1] and y[1:], which can be computed as np.dot(x[:-1], y[1:]). Analogously, the second term of the sum can be written as np.dot(x[1:], y[:-1]).

You may find out that vectorisation is not always faster than using regular for loops. The reason for this is overhead - `numpy` first needs to convert the data to temporary arrays, and when the number of the array elements is small, the time to do this would be significant compared to the calculation itself. For large arrays, however, vectorisation will well outperform for loops.

In Lecture 4, you will learn different methods to compute the integral of functions. One example is the trapezoid rule, in which you need to compute the area of several infinitesimal trapezoids to get the integral of a function. You could use an analogous vectorized function as the one in this exercise to make your code more efficient.

5. **Binary operators**
For this exercise, it is convenient to visualize the input and results both in decimal and binary notation. In `python` you can use the built-in function `bin()` to print your results in binary format.

   (a) An integer number n can be divided by 2 when its least significant digit is 0. Similarly, it is divisible by 4 when its two least significant digits are 0, etc. Write a code to check whether n is divisible by 2^m, with $m > 0$. The solution must use the binary operator &. *(Hint: first create the appropriate binary mask)*

   (b) Solve the same problem using only the shift-left (<<) and shift-right (>>) operators.

   (c) An identity operation is such that when applied to a number it produces the same number. Which are the identities for the | (bitwise OR) and for the & (bitwise AND) operators? Produce an example for each. Can you find an equivalent for the ˆ(XOR) operator (you may need more than one XOR)?

Solution:

(a) Because the & operator returns 1 only when both bits are 1, we can create a *mask* of as many 1's as the power of 2 that we want to test, and compare that to the input number. The mask can be created e.g., as $(1 << m) - 1$ (note the -1). We can then test through an `if` $(n\&((1 << m) - 1)) == 0$: clause.

(b) We can compare the input value to the result of shifting right m times followed by a shift left m times.

(c) Assuming the input value has m bits, the identity term for | is 0, for & it is $1 << (m+1)-1$ (i.e., a series of m bits all set to 1). Note that OR- or AND-ing a value with itself will also result in the input value as well. For XOR, we have n ˆ$0 = n$, but also n ˆm ˆ$m = n$.

6. **Akima interpolation**
Write an Akima sub-spline interpolator. Recall that if the known "knots" are (x_i, y_i), then the slope of the straight line segment between (x_i, y_i) and (x_{i+1}, y_{i+1}) is p_i, and the derivative of spline segment i at x_i must satisfy:

$$y'(x)|_i (x_i) = \frac{w_{i+1}p_{i-1} + w_{i-1}p_i}{w_{i+1} + w_{i-1}},$$

with weights $w_i \equiv |p_i - p_{i-1}|$. See the lecture slides for the edge constraints. Similar to the regular cubic spline, solving for the four cubic polynomial coefficients in each interval is done by imposing the constraints that the segments and their first derivatives (given above) are continuous.

(a) Test your interpolator on the lecture example in slide 23 with

```
x_data = np.array([-2, 0, 2, 4, 6, 8, 10, 12])
y_data = np.arctan(x_data - 5)
```

(b) Apply your interpolator to the data in the first row of the image of exercise 3.

Solution:

(a) The coefficients of the cubic polynomials are determined by the continuity of the first and second derivatives, y_i' and y_i'', of the sub-spline:

$$y_i(x_i) = y_i, \quad y_i(x_{i+1}) = y_{i+1}, \quad y_i'(x_i) = y_i', \quad y_i'(x_{i+1}) = y_{i+1}', \tag{5}$$

where $y_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$.

The coefficients $\{a_i, b_i, c_i, d_i\}$ are calculated by inserting $y_i(x)$ into conditions 5:

$$a_i = y_i, \quad b_i = y_i', \quad c_i = \frac{p_i - 2y_i' - y_{i+1}'}{x_{i+1} - x_i}, \quad d_i = \frac{y_i' + y_{i+1}' - 2p_i}{(x_{i+1} - x_i)^2}, \tag{6}$$

where $p_i = (y_{i+1} - y_i)/(x_{i+1} - x_i)$ is the slope between knots i and $i + 1$.

The only remaining part is the weighted derivatives (see Lecture):

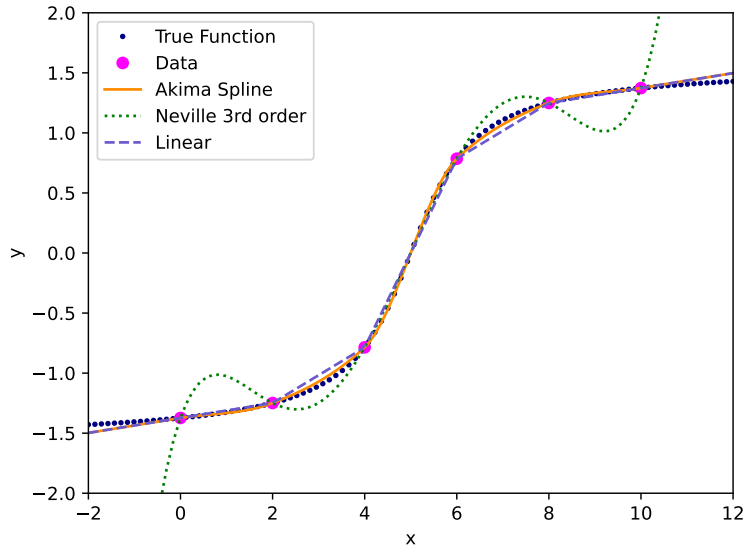
$$y_i = \frac{w_{i+1}p_{i-1} - w_{i-1}p_i}{w_{i+1} + w_{i-1}}, \quad \text{if} \quad w_{i+1} + w_{i-1} \neq 0,$$

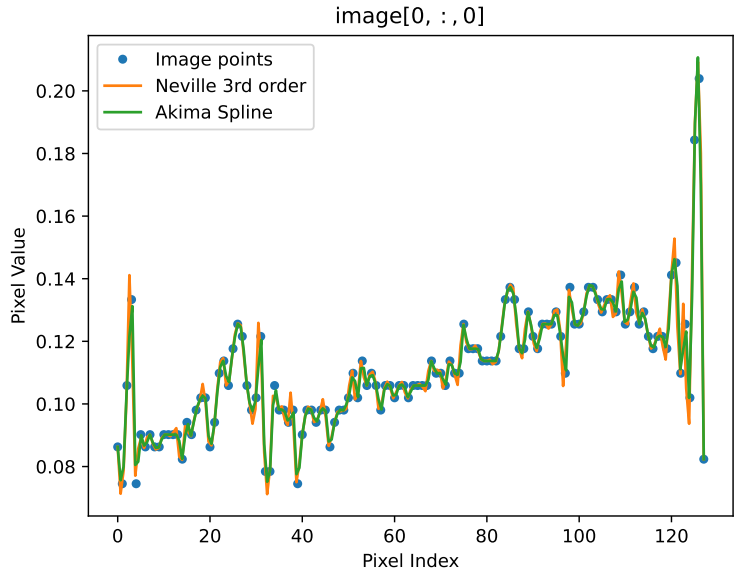
$$y_i = \frac{p_{i-1} + p_i}{2}, \quad \text{if} \quad w_{i+1} + w_{i-1} = 0,$$

where $w_i = |p_i - p_{i-1}|$.

At the endpoints:

$$y_0' = p_0, \quad y_1' = \frac{p_0 + p_1}{2}, \quad y_{n-1}' = p_{n-1}, \quad y_{n-2}' = \frac{p_{n-1} + p_{n-2}}{2}.$$




image[0, : , 0]
Image points
Neville 3rd order
Akima Spline
Pixel Value
Pixel Index
0.08
0.10
0.12
0.14
0.16
0.18
0.20
0
20
40
60
80
100
120

(b)